

VOCABULARY MAP

Concept	Kafka	RabbitMQ
Destination	topic (partitions)	queue (via exchange)
Unit of parallelism	partition	queue / stream
Read position	offset (committed)	delivery tag (ack)
Consumer unit	consumer group	competing consumers
Record-level fanout	share group (KIP-932)	—
Server process	broker	node
Metadata store	KRaft controller quorum	Raft (quorum queues)
Routing decision	client-side partitioner	broker-side exchange
Positive ack	offset commit	basic.ack
Negative ack	(none — re-deliver group)	basic.nack / reject
Producer durability	acks /ISR	publisher confirms
Dup suppression	idempotent producer	idempotent consumer
Poison-record guard	share delivery-count / limit	queue delivery-limit / DLX

DELIVERY SEMANTICS

Concern	Kafka	RabbitMQ
Producer ack	acks=0/1/all + min.in-sync.replicas	publisher confirms (confirm-Select)
Dup on retry	enable.idempotence (default true)	none broker-side; idempotent consumer
Atomic multi-write	transactional.id	no cross-queue tx
Uncommitted reads	isolation.level= read_committed	n/a — only routed messages are seen
Consumer ack	auto/manual offset commit	basic.Ack/Nack/Reject, or autoAck
Replication	ISR, page-cache flush	quorum queues, Raft + fsync

Effectively-once = at-least-once + dedup (idempotent producer/consumer). Ordering is local to one partition or one queue, never global.

KAFKA VS. RABBITMQ — DECISION

Shape	Lean
Task queue / RPC / priority / IoT-MQTT	RabbitMQ
Long retention, compaction, streams ecosystem	Kafka
Replayable fire-hose, exactly-once pipeline	Either

TTL, priority, delayed delivery, native DLX → RabbitMQ only. Log compaction, tiered storage, Kafka Streams/Connect → Kafka only. Many systems run both, one per workload.

KAFKA ARCHITECTURE

- **Topic** = named append-only log, split into ≥1 **partitions**.
- **Partition** = unit of ordering & parallelism; consumers in one group ≤ partition count.
- **Offset** = per-partition monotonic position; never reused.
- **Segment** = closed log file, rolled at segment.bytes (1 GiB) or segment.ms.
- **ISR** = replicas caught up within replica.lag.time.max.ms; leader is the only replica read/written.
- **KRaft** = controller quorum (Raft) replacing ZooKeeper; one **active controller** writes the __cluster_metadata log, followers replicate it.
- cleanup.policy=delete (age/size, retention.ms/retention.bytes) vs. compact (keep last value per key; delete via a null-value tombstone).

PRODUCER KEY PROPERTIES

Property	Meaning
acks	0 / 1 / all durability level
batch.size, linger.ms	accumulator batching window
compression.type	none/gzip/snappy /lz4/zstd
enable.idempotence	dedup retries (needs acks=all)
max.in.flight.requests.per.connection	≤5 with idempotence on
delivery.time.out.ms	upper bound incl. retries (120s)
transactional.id	enables the transactional producer

CONSUMER KEY PROPERTIES

Property	Meaning
group.id	consumer group name
enable.auto.commit	periodic vs. manual offset commit
auto.offset.reset	latest / earliest / none
max.poll.records	records per poll() (500)
max.poll.interval.ms	liveness bound (5 min)
isolation.level	read_uncommitted / read_committed

subscribe() = group-coordinated; assign() = manual, no rebalance. seek() / seekToBeginning() / seekToEnd() reposition owned partitions.

SHARE-CONSUMER (KIP-932)

vs. consumer group	Share group
Ownership	any member, any partition
Ack unit	per record, 4 outcomes
Redelivery	only that record

ACCEPT done · RELEASE retry now · REJECT permanent fail · RENEW extend lock.

group.share.record.lock.duration.ms (30s) · group.share.delivery.count.limit (5) archives poison records · share.acknowledgement.mode implicit/explicit · share.acquire.mode batch_optimized/record_limit.

RABBITMQ AMQP MODEL

Exchange	Routing rule
direct	binding key == routing key
fanout	ignores key, all bound queues
topic	pattern match, * =1 word, # =0+ words
headers	matches header table, x-match any/all

Nameless **default exchange** = direct, queue name as binding key. Also: x-consistent-hash (sharding by weight), x-local-random (same-node routing, 4.0+). x-delayed-message is a 4.3-archived community plugin.

QUEUE PROPERTIES & TYPES

Property	Meaning
durable	definition survives restart
exclusive	one connection, auto-deleted on close
auto-delete	removed after last consumer leaves
arguments	x-queue-type, x-message-ttl, x-max-length, x-dead-letter-exchange, ...
Type	Trait
classic	single node, not replicated, lowest latency
quorum	Raft-replicated, always durable, delivery-limit 20
stream	append-only replicated log, replayable, filters

ACK / CONFIRM ONE-LINERS

Kafka producer — async send(), no ack until acks is satisfied; idempotence dedups retries broker-side.

Kafka consumer — commitSync() / commitAsync() after processing = ack-after (at-least-once); auto-commit before = ack-before (at-most-once).

RabbitMQ publisher — confirmSelect() then waitForConfirms() or addConfirmListener.

RabbitMQ consumer — basicAck/basicNack/basicReject on a delivery tag; autoAck=true is unsafe for anything that must not be lost.

DOCKER ONE-LINERS

```
docker run -p 9092:9092 apache/kafka:latest

docker run -it --rm --name rabbitmq \
  -p 5672:5672 -p 15672:15672 \
  rabbitmq:4-management
```

Kafka: single-node KRaft (broker+controller). RabbitMQ: 5672 AMQP, 15672 management UI (guest/guest, localhost only).

KAFKA CLI

Tool	Purpose
kafka-topics.sh	create/describe/delete topics
kafka-consumer-groups.sh	describe lag, reset offsets
kafka-configs.sh	dynamic config, any entity type
kafka-console-producer/consumer.sh	ad hoc produce/consume
kafka-features.sh	describe/upgrade feature versions

```
bin/kafka-topics.sh --bootstrap-server \
  localhost:9092 --create --topic orders \
  --partitions 12 --replication-factor 3

bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:9092 \
  --describe --group order-processors
```

RABBITMQ CLI

Tool	Purpose
rabbitmqctl	users, vhosts, permissions, policies
rabbitmqadmin	HTTP API client (v2, JSON)
rabbitmq-diagnostics	read-only health checks
rabbitmqctl set_permissions -p payments \ order-service "^orders\" \"^orders.\" \ \"^orders.\" \ \"^orders\".	
rabbitmqadmin declare queue \ --vhost=payments name=orders durable=true	
rabbitmq-diagnostics check_port_connectivity	

OPS CONFIG KNOBS

Kafka	RabbitMQ
retention.ms / bytes	x-message-ttl
cleanup.policy	x-overflow
min.insync.replicas	x-dead-letter-exchange
replica.lag.time.max.ms	x-single-active-consumer

Kafka: kafka-configs.sh —alter changes live config with no restart. RabbitMQ: policies (rabbitmqctl set_policy) apply queue arguments cluster-wide by pattern.

PROTOCOL → SPRING CLASS

Role	Kafka	RabbitMQ
Send	KafkaTemplate	RabbitTemplate
Receive	@KafkaListener	@RabbitListener
Admin	KafkaAdmin	AmqpAdmin
Factories	Consumer/Producer Factory	CachingConnectionFactory
Request/reply	ReplyingKafkaTemplate	direct reply-to, @SendTo
Share consumer	ShareKafka — aListener Container Factory	
Tx	KafkaTransactionManager	channel tx (rare)
Cloud Stream	StreamBridge, Supplier/Function/Consumer	

Kafka has **no** dedicated Boot starter — add plain spring-kafka. RabbitMQ uses spring-boot-starter-amqp.

SPRING. KAFKA.*

bootstrap.servers	broker list
consumer.properties.	new consumer
group.protocol	group protocol
consumer.isolation-level	read_committed / uncommitted
listener.ack-mode	container-wide ack mode
streams.application-id	Kafka Streams app id

Since Spring for Apache Kafka 4.1, @KafkaListener(ackMode=...) overrides the container-wide listener.ack-mode per listener.

SPRING. RABBITMQ.*

host / address.es	broker connection
virtual-host	vhost to connect to
listener.type	simple / direct
listener.simple.acknowledge-mode	auto / manual / none
ssl.enabled	enable TLS
template.reply-timeout	RPC reply wait

SPRING. CLOUD.STREAM.*

bindings.<name>.destination	topic/exchange to bind
bindings.<name>.group	consumer group name
bindings.<name>.consumer.max-attempts	retry before DLQ
kafka.bindings.<name>.*	Kafka binder-specific
rabbit.bindings.<name>.*	RabbitMQ binder-specific

Binding names derive from the functional bean: <bean>-in-0 / -out-0. spring.cloud.function.definition selects which bean(s) bind.